MODERN IMAGE PROCESSING

CHAPTER 4 • PART 1 OF 2

Multiresolution Image Representations

Why Multiple Resolutions? → Pyramid Reconstruction

Lecture Notes

In This Part

01 Why Multiple Resolutions?

02 Image Downsampling

03 Smoothing Before Downsampling

04 The Gaussian Pyramid

05 Image Upsampling

06 The Laplacian Pyramid

07 Pyramid Reconstruction

CHAPTER OVERVIEW

Chapter Overview

- Chapter 3 ended on a question: instead of processing one neighborhood at a time, can we represent an image at multiple resolutions?
- This chapter answers it, and the answer turns out to be one of the central constructions of image processing.
- Starting from nothing but the Gaussian smoothing of Chapter 3 and the subtraction trick of unsharp masking, we build the Gaussian pyramid, then the Laplacian pyramid, and then — by asking one more question at exactly the right moment — the Haar transform, the first true image transform of this book and the direct realization of the "good basis" promised in §1.7.
- The chapter ends where the transform's own structure points: at compression, and at a question that motivates the whole next chapter.
- Throughout, we keep the promise made in Chapter 3: no frequency-domain language, no filter-bank theory — everything is averages, differences, and sliding windows.

SECTION 1

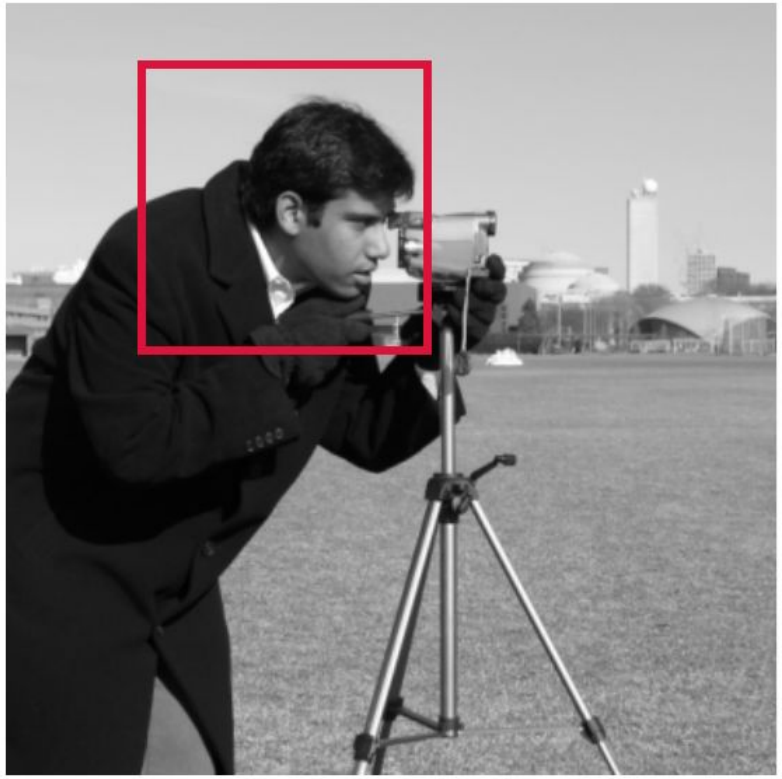
Why Multiple Resolutions?

Why Multiple Resolutions?

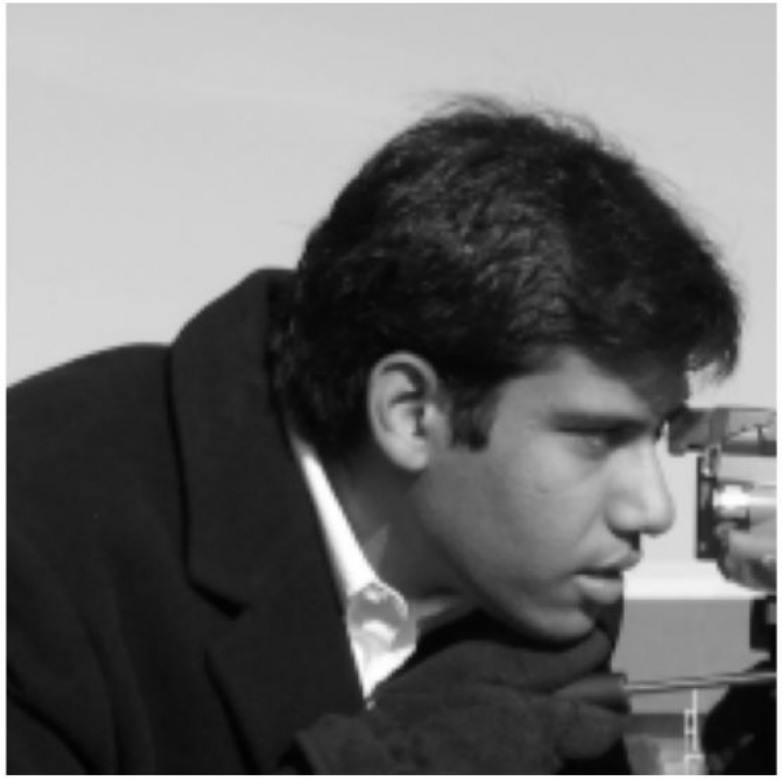
- A photograph is not organized at one scale. the figure makes the point with the cameraman: the picture simultaneously contains large structures — the dark figure against the bright sky, the sweep of the field — and fine structures — individual hairs, the camera's dials, blades of grass.
- Both are real image content; they simply live at different sizes.
- Chapter 3 taught us that any single filter commits to one scale, its window size, and is blind to the others: the 3×3 window that finds fine edges cannot see the figure/sky boundary as a whole, and the wide Gaussian that captures the large shapes obliterates the hair.

Why Multiple Resolutions?

coarse structure: figure vs. sky,
coat, large shapes



fine structure inside the box:
hair, face, camera parts



One image, two scales of content. The large shapes (left, red box marking a region) and the fine detail inside that region (right) are both genuine structure — at sizes two orders of magnitude apart.

Why Multiple Resolutions?

- Several everyday operations run into the same mismatch:

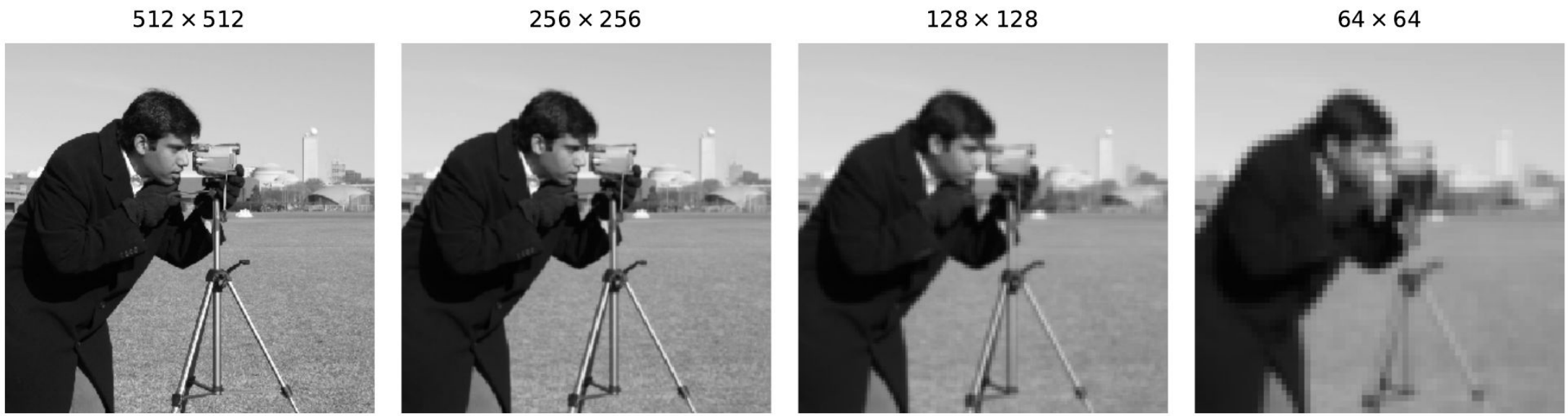
Why Multiple Resolutions?

- Zooming. A viewer at full zoom needs the fine detail; zoomed out, the same pixels are wasted — the display can only show the coarse structure anyway.
- Image transmission. Over a slow connection it is far better to deliver a coarse version immediately and refine it progressively than to deliver pixels row by row — but that requires having coarse versions to send first.
- Compression. Chapters 1 and 2 argued that compression lives in representations that concentrate content into few numbers. Coarse structure is, plausibly, describable with few numbers — if we can separate it from the detail.
- Analysis. Finding an object of unknown size (a face, a car) is natural if the image is available at many sizes: search each version with one fixed-size window.

Why Multiple Resolutions?

- All four point the same way.
- Rather than one image at one resolution, compute a sequence of versions of the image — full resolution, half, quarter, eighth — each a legitimate summary of the scene at its own scale (the figure).
- Stacked smallest-on-top, the sequence looks like a pyramid, and image pyramid is its name.
- Building one correctly is the business of the next three sections; it is less trivial than it sounds.

Why Multiple Resolutions?



The same image at four resolutions, displayed at the same physical size: 512^2, 256^2, 128^2, 64^2. Fine detail fades level by level; the large structures survive to the coarsest version.

SECTION 2

Image Downsampling

IMAGE DOWNSAMPLING

Decimation

- To halve an image's resolution, the obvious operation is decimation: keep every second pixel in each direction and discard the rest,
- turning a 512×512 image into 256×256 — one quarter of the pixels.
- Repeating gives 128^2, 64^2, and so on.
- The information loss is deliberate and quantifiable: each decimation discards three of every four pixels, and no operation can later recover what was thrown away.
- The entire question of this section is whether the quarter that remains is a faithful coarse summary of the image.

$$J_{ij} = I_{2i,\,2j},$$

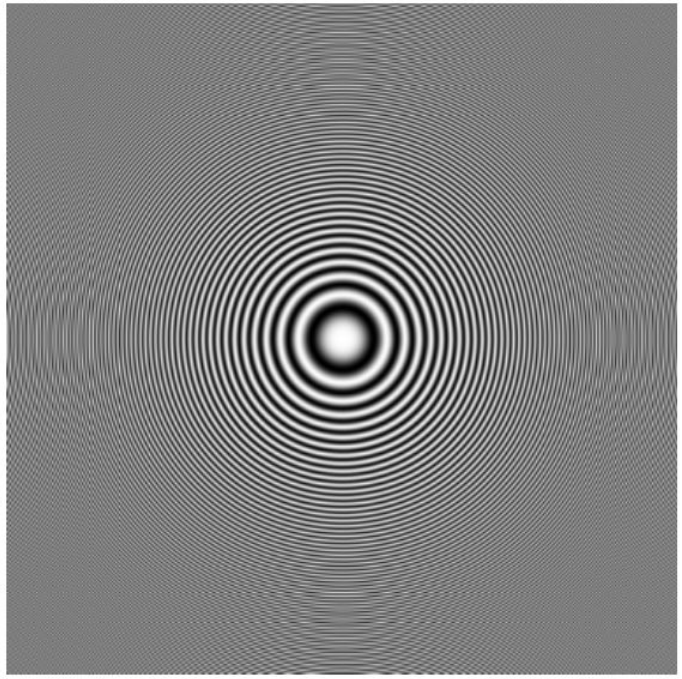
Aliasing: when decimation lies

- For smooth, slowly varying content it is: neighboring pixels are nearly equal (the redundancy of §1.7 once again), so keeping one in four loses little.
- The trouble is content that varies quickly — structure finer than the new pixel spacing can represent.
- Such structure does not vanish gracefully under decimation; it does something far worse: it masquerades as coarse structure that was never in the scene.
- The phenomenon is called aliasing, and we introduce it, as promised, purely visually.

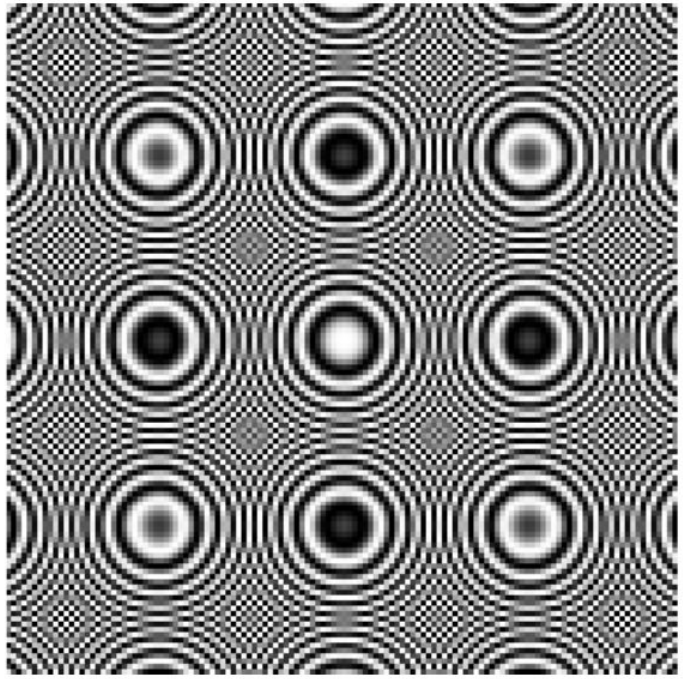
IMAGE DOWNSAMPLING

Aliasing: when decimation lies

test pattern: rings get finer
toward the edge



keep every 4th pixel:
false coarse rings appear (aliasing)



photo, every 4th pixel:
jagged edges, broken tripod lines



Aliasing. Left: a test pattern of concentric rings, growing finer toward the edges. Center: keeping every fourth pixel — the fine outer rings reappear as bold, slow false ring patterns centered where no rings were. Right: the same decimation on the photograph — edges turn jagged, and the tripod's thin legs break into dashes.

IMAGE DOWNSAMPLING

Aliasing: when decimation lies

- the figure shows the classic demonstration.
- The test pattern's rings grow finer toward the border; after decimation, the fine rings have not simply blurred away — they have reorganized into entirely fictitious coarse rings, as prominent as the true ones.
- The mechanism can be felt without any theory: sampling a fast oscillation at too coarse a step lands on a misleading subsequence of its values — like a film of a spinning wheel that seems to rotate slowly backward because each frame catches the spokes near their previous positions.
- The samples are individually correct; the pattern they form is a lie.
- On the photograph the same effect is less flamboyant but just as damaging: smooth diagonal edges become staircases, one-pixel-wide structures (tripod legs, wires) intersect the surviving sampling grid only intermittently and break into dashes, and fine texture turns into gritty noise.

Aliasing: when decimation lies

- Direct decimation is therefore not a valid way to build a pyramid level: the coarse image it produces contains artifacts posing as content, and every subsequent computation on the pyramid would inherit them.
- The diagnosis suggests its own cure.
- Aliasing is caused by structure too fine for the coarse grid to represent.
- If that structure is going to corrupt the result, it must be removed before sampling — and removing fine structure is precisely what Chapter 3 taught us to do.

SECTION 3

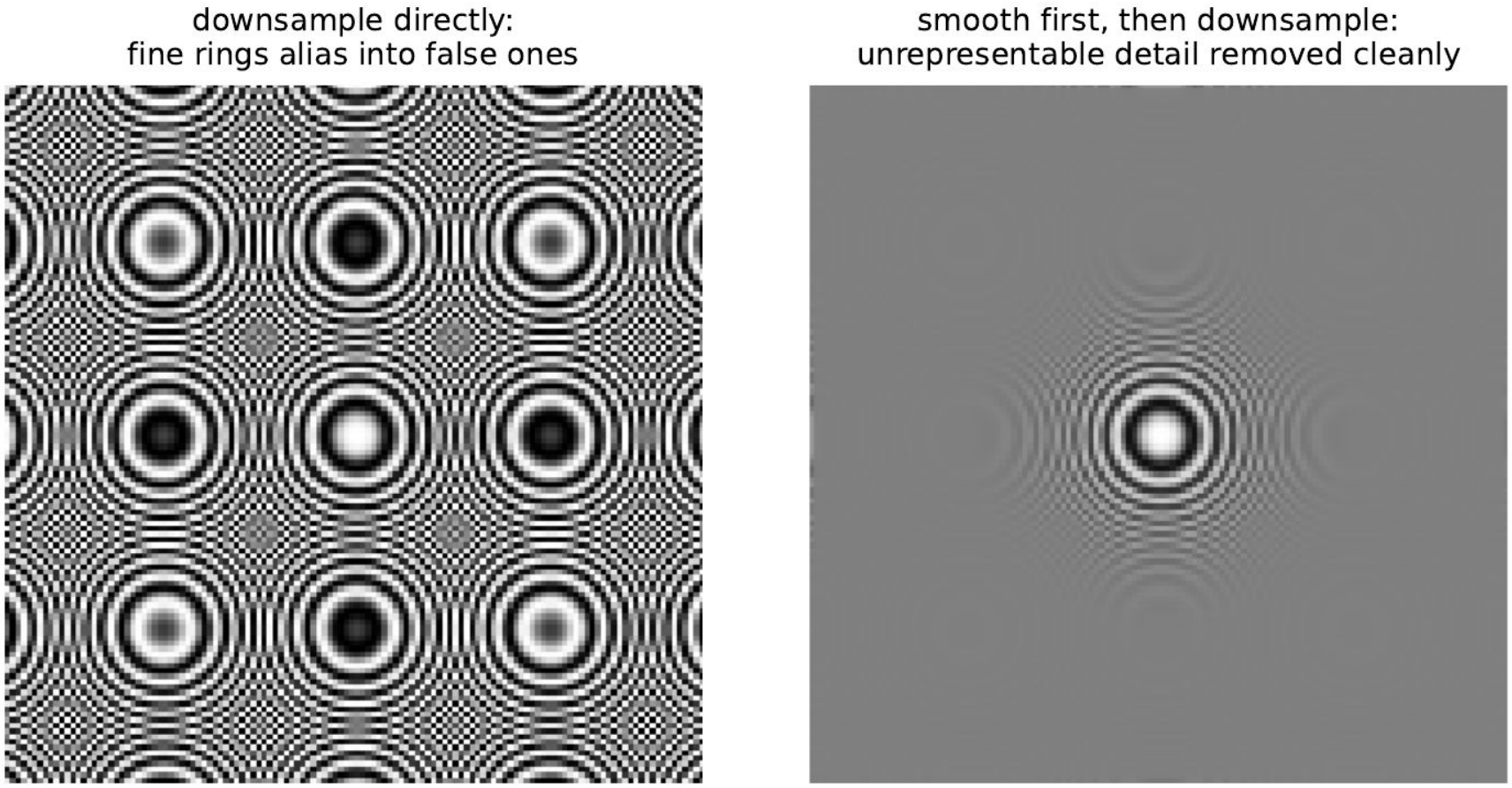
Smoothing Before Downsampling

Smoothing Before Downsampling

- The fix is one Gaussian filter (§3.5) applied before decimation:
- Smoothing removes exactly the fine-scale variation the coarse grid cannot carry; what remains varies slowly enough that every-second-pixel sampling represents it honestly.
- The smoothing width is matched to the decimation factor — enough blur that nothing finer than the new pixel spacing survives, and no more, so that all representable structure passes through.
- (For the factor-of-2 steps used in pyramids, a Gaussian with σ ≈ 1 is the standard choice; for the factor-of-4 decimation in the figures, proportionally more.)

$$J = \big(\text{Gaussian-smooth } I\big)\big[::2,\ ::2\big] \qquad \text{(smooth first, then keep every second pixel).}$$

Smoothing Before Downsampling



The same factor-4 downsampling of the ring pattern, without (left) and with (right) prior Gaussian smoothing. Smoothing first removes the unrepresentable fine rings cleanly: they fade to flat gray instead of aliasing into false structure. Coarse rings, which the small grid can represent, pass through intact.

Smoothing Before Downsampling

- the figure shows the payoff on the worst case: with smoothing, the unrepresentable outer rings fade to featureless gray — honestly absent — while the representable inner rings survive untouched.
- The information in the fine rings is still lost (it had to be; the small image has nowhere to put it), but it is lost silently instead of being converted into false content.
- That is the entire principle: downsampling must be preceded by smoothing matched to the sampling step.
- We offer no frequency-domain justification — none is needed; the two figures are the argument — and move straight to using the principle.

SECTION 4

The Gaussian Pyramid

Construction

- Everything is now assembled.
- Define the reduce operation:
- — one correct halving of resolution.
- The Gaussian pyramid of an image I is the sequence obtained by applying reduce repeatedly:

$$\text{REDUCE}(I) = \text{Gaussian-smooth } I, \text{ then keep every second pixel in each direction,}$$

$$G_0 = I, \qquad G_{k+1} = \text{REDUCE}(G_k), \qquad k = 0, 1, 2, \ldots$$

Construction

- For a 512×512 image, G_0 is the original, G_1 is 256^2, G_2 is 128^2, G_3 is 64^2, and the recursion can continue down to a single pixel (the image's overall mean) if desired; four to six levels are typical in practice.
- The name records the two ingredients: Gaussian smoothing inside reduce, and the pyramid shape of the stacked levels — each level one quarter the area of the one below, so that the entire stack above G_0 costs only $\frac{1}{4} + \frac{1}{16} + \cdots < \frac{1}{3}$ extra storage.

Construction

G_0: 512^2



G_1: 256^2



G_2: 128^2

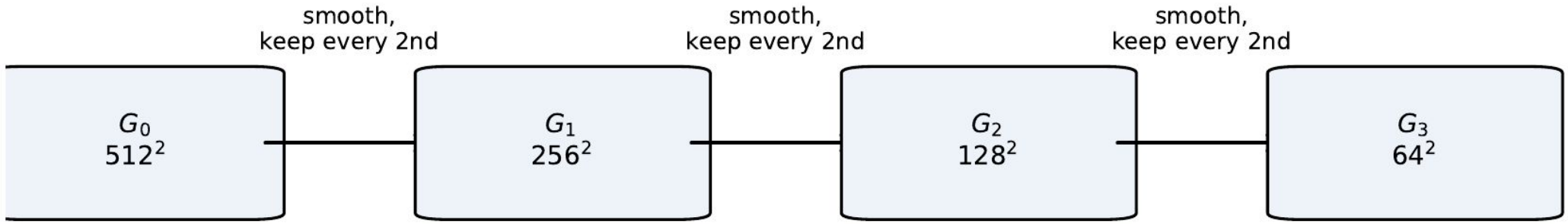


G_3: 64^2



A four-level Gaussian pyramid, levels shown at true relative sizes. Each level is a smoothed, halved version of its predecessor: a legitimate photograph of the same scene at a coarser scale.

Construction



Gaussian pyramid construction: each arrow is one reduce (smooth, then keep every second pixel).

THE GAUSSIAN PYRAMID

What each level means

- The pyramid's levels are not merely shrunken copies; each is a principled object.
- G□ is the image as seen at scale 2^k: all structure finer than 2^k pixels (in original coordinates) has been absorbed by the accumulated smoothing, and all coarser structure survives.
- Reading the figure left to right is watching the image's scales peel away in order — grass texture first, then facial detail, then the camera, until at G_3 only the composition remains: a dark figure, a horizon, a sky.
- Equivalently — recall the observation banked in §3.5 that repeated Gaussian smoothing climbs a ladder of coarser versions — the pyramid is that ladder, with each rung economically stored at the resolution it actually needs.

Applications

- The Gaussian pyramid directly serves every motivation of §4.1.
- Progressive transmission: send G_3 first (a few thousand pixels, instantly displayable), then successively finer levels.
- Multi-scale search: to find an object of unknown size, run one fixed-size detector over every level — an object too large for the window at G_0 fits it at G_2.
- Fast coarse computation: many algorithms (registration, motion estimation) run first at a coarse level, where the problem is thousands of times smaller, and refine the answer down the pyramid.
- And — the use this chapter cares about most — the pyramid is the scaffolding for a genuinely new representation, built in §4.6 from the question the pyramid begs: each reduce destroys detail; where did it go?

Applications

- To ask that question precisely, we first need the ability to travel back up in resolution.

SECTION 5

Image Upsampling

Expansion

- Downsampling has an inverse-in-shape (though not in information): upsampling, or expand — producing from a small image a larger one of the original dimensions.
- Since the small image simply does not contain the fine detail of the large one, upsampling cannot manufacture information; its honest job is to produce the best guess at the large image consistent with the small one, filling the new pixel positions by interpolation between the known ones.
- Two interpolation rules cover this book's needs:

IMAGE UPSAMPLING

Expansion

- Nearest-neighbor: assign every new pixel the value of its closest known pixel. Each known pixel becomes a solid 2×2 (or 4×4) block. Trivially fast, and the result is visibly blocky — the pixel grid itself becomes the dominant structure (the figure, center).
- Bilinear: assign every new pixel a distance-weighted average of its (up to four) surrounding known pixels, so values ramp smoothly between known samples.
- Marginally more work, and the blockiness disappears — replaced by an honest smoothness (the figure, right).
- Note carefully what bilinear interpolation does not do: the result is smooth, not sharp.
- No fine detail returns, because none was present to interpolate from.

IMAGE UPSAMPLING

Expansion



Upsampling a 128^2 pyramid level by a factor of 4. Nearest-neighbor (center) turns every sample into a block; bilinear (right) ramps smoothly between samples. Neither recovers detail — there is none to recover — but bilinear produces the more faithful prediction of the full-resolution image.

Why expansion matters here

- Within this chapter, expand (always bilinear from now on) has one crucial role: it makes images at adjacent pyramid levels comparable.
- G_2 and G_1 cannot be subtracted — they are different sizes — but expand(G_2) and G_1 can.
- And expand(G_{l+1}) has a precise interpretation: it is the prediction of G_l from one level coarser — everything about G_l that the coarse level knew, returned to G_l's resolution, with the detail lost by reduce showing up as exactly the prediction's error.
- That error is the object the next section is named after.

SECTION 6

The Laplacian Pyramid

What information was lost?

- Follow the question through.
- The step from G□ down to G□₊₁ destroyed detail; expand(G□₊₁) is the coarse level's best prediction of G□; therefore the difference
- is precisely the information that reduce lost at level k: what the fine level knows that the coarse level does not. the figure shows the three pieces.
- The difference image L_1 (right panel) should look familiar: near zero (mid-gray) in smooth regions, active exactly at edges and fine texture — it is a detail image, the same species as unsharp masking's "image minus smoothed" residue (§3.7).
- The name completes the connection: because subtracting a smoothed version acts like the second-difference filtering of §3.9, the L□ are called Laplacian images, and the construction is exactly the "image-minus-smoothed between successive smoothing levels" foretold at the end of §3.9 — now with the two levels living at different resolutions, joined by expand.

$$L_k = G_k - \text{EXPAND}(G_{k+1})$$

What information was lost?

G_1 (256^2)



expand(G_2): blurry prediction of G_1



$L_1 = G_1 -$ expand(G_2):
the lost detail



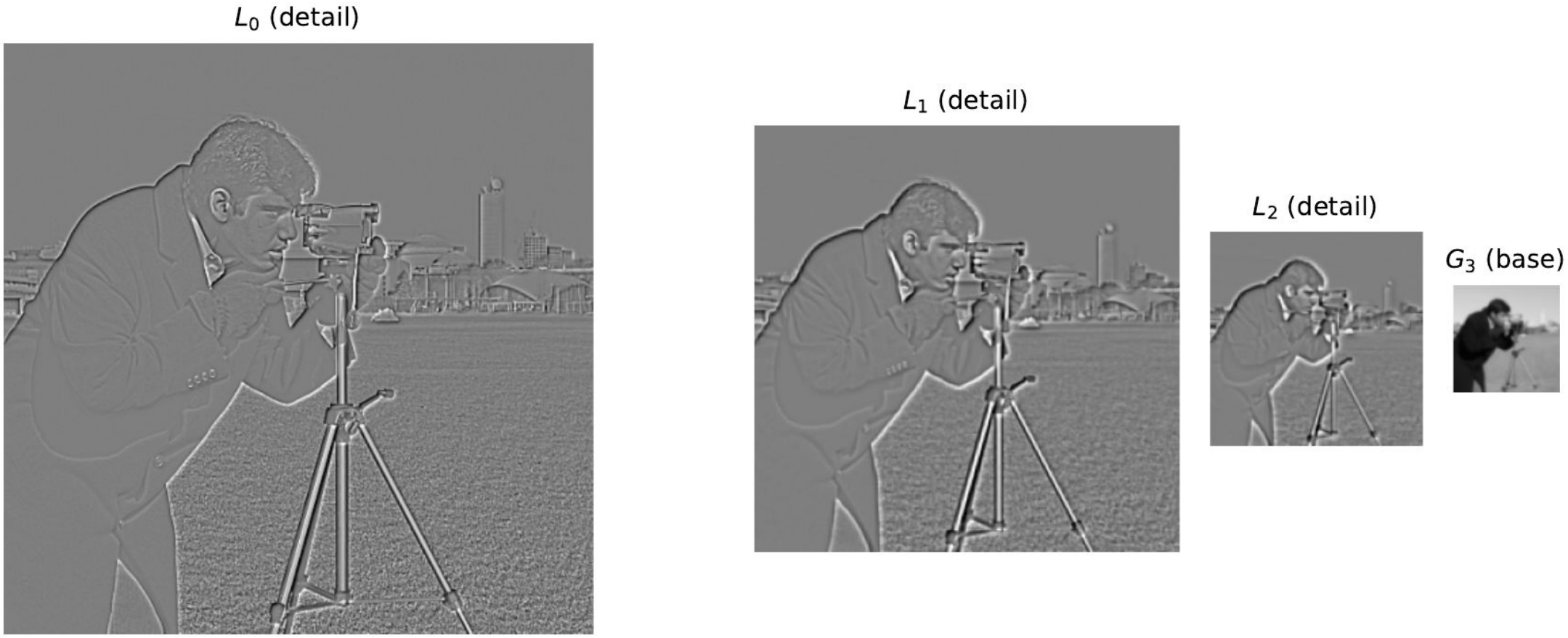
One Laplacian level. Left: G_1. Center: expand(G_2), the blurry prediction of G_1 from one level coarser. Right: their difference L_1 — displayed with mid-gray as zero — containing exactly the detail that lives at this scale: nothing smooth, everything fine.

Construction and interpretation

- Apply the subtraction at every level and keep the coarsest Gaussian image as the base:
- The sequence L_0, L_1, …, L_K is the Laplacian pyramid (the figure).
- Its interpretation is the payoff of the whole construction so far: the image has been decomposed by scale.
- L_0 holds only the finest detail (hair, grass, camera dials); L_1 the next scale up; L_2 coarser structure still; and the tiny base G_3 holds the global composition.
- Each layer contains only its own scale — the content of every other scale has been subtracted away — and together the layers hold everything, as the next section makes exact.

$$L_k = G_k - \text{EXPAND}(G_{k+1}) \quad (k = 0, \ldots, K-1), \qquad L_K = G_K.$$

Construction and interpretation



The Laplacian pyramid: three detail layers (mid-gray = zero) at successively coarser scales, plus the coarse Gaussian base. The image has been separated into "what lives at each scale."

Construction and interpretation

- Notice, with an eye on §4.10, a striking property visible in the figure: the detail layers are mostly mid-gray — mostly zero.
- Smooth regions, which dominate natural images, contribute nothing to any detail layer.
- The pyramid has quietly concentrated the image's content into a minority of its numbers: the sparsity promised by §1.7 and §2.10, appearing here for the first time in a concrete representation.

SECTION 7

Pyramid Reconstruction

PYRAMID RECONSTRUCTION

Nothing was lost

- The Laplacian pyramid was built by subtraction, and subtraction is reversible by addition.
- Each construction equation
- which reads: any Gaussian level is recoverable from the next-coarser level plus this level's detail.
- Since the coarsest level G_K = L_K is stored outright, the whole image follows by walking the recursion upward:

$$L_k = G_k - \text{EXPAND}(G_{k+1}) \qquad \text{rearranges to} \qquad G_k = L_k + \text{EXPAND}(G_{k+1}),$$

$$G_K = L_K \longrightarrow G_{K-1} = L_{K-1} + \text{EXPAND}(G_K) \longrightarrow \cdots \longrightarrow G_0 = L_0 + \text{EXPAND}(G_1).$$

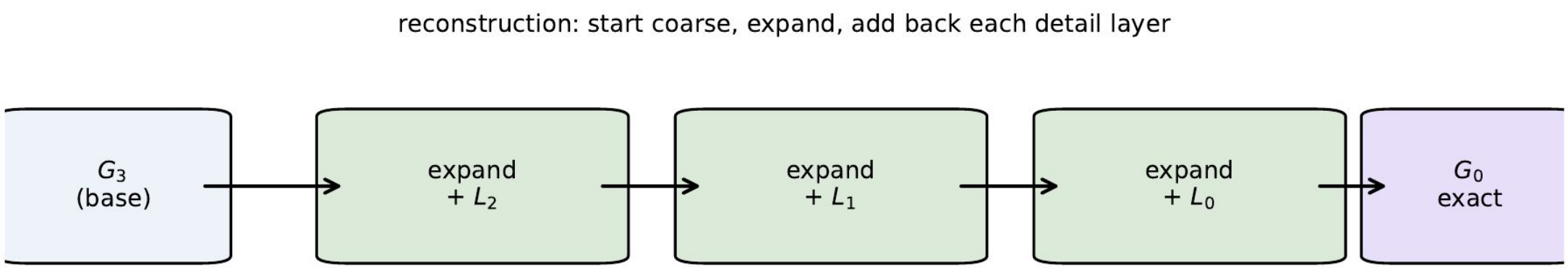
PYRAMID RECONSTRUCTION

Nothing was lost

- Start from the 64^2 base; expand it and add back L_2's detail to obtain G_2 exactly; expand and add L_1 to obtain G_1; expand and add L_0 — and the result is G_0, the original image, exactly.
- Not approximately: every subtraction is undone by the matching addition, whatever imperfections expand may have, because the same expand output appears on both sides.
- (Computing the round trip numerically on the cameraman image gives a relative error of 10^{-18} — machine precision; the figures of this section are that computation.) No proof is needed beyond the rearranged equation; perfect reconstruction is built into the construction.

PYRAMID RECONSTRUCTION

Nothing was lost



Reconstruction pipeline: start from the coarse base, repeatedly expand and add back the detail layer of each scale.

PYRAMID RECONSTRUCTION

Nothing was lost

start: G_3 (64^2)



$+L_2 \rightarrow 128^2$



$+L_1 \rightarrow 256^2$



$+L_0 \rightarrow 512^2$



Stepwise reconstruction. Each panel adds one detail layer to the expanded previous panel: composition first, then coarse structure, then mid-scale content, then fine detail — until the original image is recovered exactly.

PYRAMID RECONSTRUCTION

Nothing was lost

- the figure shows the walk visually, and it doubles as a demonstration of progressive transmission from §4.1: each panel is exactly what a receiver could display after each installment of data — the image sharpening scale by scale as detail layers arrive.

PYRAMID RECONSTRUCTION

The representation, appraised

- Step back and take stock, because the Laplacian pyramid is this book's first complete alternative representation of an image.
- It is invertible: original pixels in, layers out, original pixels back — a change of description with no information loss, exactly the standard §1.7 set for a basis representation.
- It is organized by scale: each stored number belongs to an identifiable level of detail, something raw pixels cannot claim.
- And it is nearly sparse: most stored values sit near zero.
- One blemish keeps it from perfection: it is slightly redundant — the layers together hold 43 as many numbers as the image ($512^2 + 256^2 + \cdots$), a 33% overhead.

The representation, appraised

- For a representation whose purpose is to concentrate information, storing extra numbers is an irony worth removing.
- The removal is the subject of the rest of the chapter, and it begins with a one-page thought.

KEY TAKEAWAYS

Multiresolution Image Representations

- Why Multiple Resolutions?
- Image Downsampling
- Smoothing Before Downsampling
- The Gaussian Pyramid
- Image Upsampling
- The Laplacian Pyramid
- Pyramid Reconstruction

End of Part